

AI-FitTrack

AI-Augmented Fitness Tracking — Backend RESTful API
Project Documentation

FitTrack AI is a scalable RESTful backend service built on Node.js, Express.js, MongoDB Atlas, and the Google Gemini AI SDK following the Model-View-Controller (MVC) architecture. Designed as the primary technical overview for developers, this documentation details the system's API endpoints, database schemas, and security infrastructure. The platform ensures end-to-end data privacy using bcrypt password encryption and JSON Web Token (JWT) session authorization. Core features include complete CRUD operations for workout tracking, complemented by real-time search filtering across exercise history. By integrating artificial intelligence, the backend processes user demographics and historical activity to generate personalized workout recommendations and performance insights. Built for production reliability, the API incorporates centralized error handling, modular routing, and standardized JSON responses ready for seamless client integration.

TEAM DETAILS:

TEAM ID: SWTID-2026-3254

Register Number	Team Members	Email-id
24UCS251011	Gowtham S	mr.gowtham1111@gmail.com
24UCS251009	Giridharan C	giritharan9842@gmail.com
24UCS251010	Gnanagiri M	gnnagiri@gmail.com
24UCS251012	Hariharasudhan R	hariharasudhanrs29@gmail.com

BRAINSTORMING & IDEATION PHASE

1. Problem Identification & Need Analysis

Scenario / Background

- Hari hasasudhan is an active fitness enthusiast who maintains consistency in his exercise routine but relies on manual notebooks and static spreadsheets to track his daily workouts.
- Over time, managing physical logs becomes burdensome, making it hard to organize workout history, evaluate progress over weeks or months, or determine if his routine aligns with his long-term fitness goals.
- Additionally, he lacks access to professional fitness coaching to adapt his workout routines according to his evolving fitness level.

Core Inefficiencies Identified

- Centralization Issues: Workouts are fragmented across paper notes or disconnected spreadsheets, making unified record management difficult.
- Calculation Errors: Manual calculation of total exercise duration and calories burned is time-consuming and prone to human error.
- Lack of Secure User Authentication: Personal health and activity records lack password encryption and protected user access.
- Inefficient Record Search: Users cannot search or filter past workout history by date, category, or exercise name efficiently.
- Absence of Personalized Guidance: Traditional tracking offers static history without dynamic recommendations or qualitative performance feedback.

2. Proposed AI-Augmented Solution

Centralized RESTful API Backend

- A scalable RESTful application built with Node.js, Express.js, MongoDB, and Mongoose following the MVC architecture.
- Implements JSON Web Tokens (JWT) for route protection and bcrypt.js password hashing for secure authentication and user privacy.
- Provides full CRUD functionality and search capabilities for managing workout records securely.

Google Gemini AI Integration

- Demographic-Based Tailored Workout Plans:
 - Inputs: Age, Fitness Goal, and Experience Level.
 - AI Outputs: Weekly exercise plans, tailored recommendations (Beginner / Intermediate / Advanced), safety guidelines, and motivational advice generated via Google Gemini AI.
- Performance Insights via Activity Stats:
 - Inputs: Total Workouts completed, Average Workout Duration, and Total Calories Burned.
 - AI Outputs: In-depth performance summaries, progress evaluation, and actionable advice to optimize training efficiency.

3. Target Audience & Social Impact

Target Audience / Primary Beneficiaries

- Fitness Enthusiasts: Individuals who exercise regularly and need centralized tracking and advanced performance insights.
- Beginners: Users who lack personal trainers and require safe, structured, level-appropriate exercise routines.
- Trainers & Coaches: Professionals seeking standardized data formatting to monitor client activity and progress.

Societal Value & Social Impact

- Democratizing Fitness Guidance: Replaces expensive human personal training with automated, high-quality, AI-driven guidance accessible to anyone.
- Data Integrity & Privacy: Protects personal health records using industry-standard security protocols while eliminating manual logging errors.
- Encouraging Long-Term Health Consistency: Provides structured tracking and intelligent feedback that keeps individuals motivated and safe during physical training.

REQUIREMENT ANALYSIS PHASE

Requirement Analysis Phase

1. Software Requirements

The FitTrack AI backend requires the following runtime ecosystems, framework libraries, database drivers, and development tools:

- Operating System: Windows 10/11, macOS, or Linux (cross-platform support).
- Runtime Ecosystem: Node.js (v16 or above) with npm (v8 or above) for dependency management.
- Web Framework: Express.js for RESTful routing and middleware gateways.
- Database & ODM: MongoDB (NoSQL document store) with Mongoose (Object Data Modeling).
- Core Dependencies: dotenv (environment variables), bcryptjs (password hashing), jsonwebtoken (JWT authentication), cors (cross-origin sharing), and @google/genai (Gemini API integration).
- Development & Testing Tools: Visual Studio Code, Postman / Thunder Client (API testing), Git / GitHub (version control).

2. Hardware Requirements

- Processor: Intel Core i3 (8th Gen or above) / AMD Ryzen 3 or equivalent.
- RAM: 4 GB minimum (16 GB recommended for concurrent instances of MongoDB and application servers).
- Storage: 1 GB available disk workspace.

3. Functional Requirements

3. Functional Requirements

- **FR1: User Authentication & Authorization**
 - **Registration & Login:** Securely onboard users by validating identity credentials, encrypting passwords, and checking login attempts against stored records.
 - **Token Generation:** Issue signed JSON Web Tokens (JWT) upon successful login to handle stateless user sessions and profile access.
- **FR2: Workout Management (CRUD Operations)**
 - **User Data Association:** Link every workout record directly to the authenticated user's unique ID.
 - **Activity Tracking:** Allow users to log new sessions (workout title, category, duration, calories burned, date), view their full history, and remove past records.
- **FR3: Dynamic Search & Filtering**
 - **Query Handling:** Allow users to filter historical workouts using keyword matching on workout names (case-insensitive) or by selecting specific exercise categories.
- **FR4: AI Workout Recommendation**
 - **Personalized Plans:** Feed demographic and training parameters (age, fitness targets, experience level) into Google Gemini AI.

- **Output Delivery:** Return custom weekly exercise routines, targeted training advice, and injury prevention guidelines.

- **FR5: AI Fitness Insights**

- **Data Aggregation:** Analyze cumulative user metrics (total completed sessions, average workout length, total calories burned).
- **Actionable Feedback:** Use Google Gemini AI to evaluate progress trends and return performance evaluations with motivational advice.

4. Non-Functional Requirements

- **NFR1: Password Encryption**

- **Cryptographic Hashing:** Salt and hash all user passwords via bcrypt prior to database storage, ensuring plain-text credentials are never saved.

- **NFR2: Route Authorization Middleware**

- **Access Control:** Intercept requests to private routes, verify the HTTP Bearer token signature, reject expired or invalid tokens, and inject authenticated user data into the request pipeline.

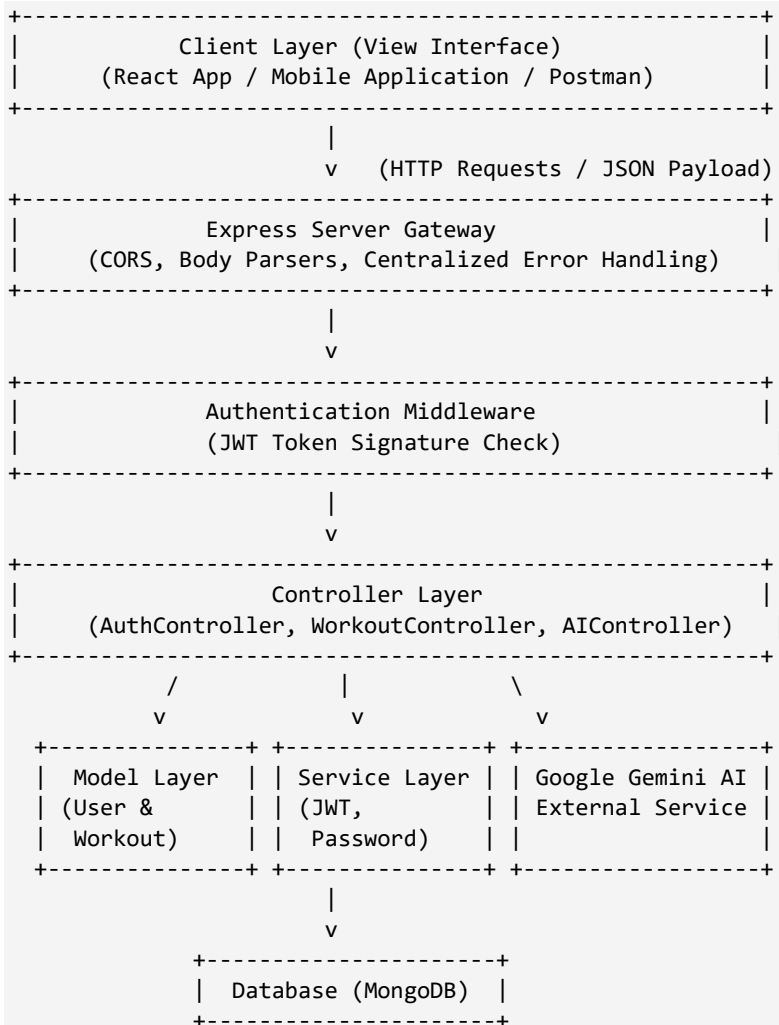
- **NFR3: JSON Standardization & Centralized Error Handling**

- **Uniform Responses:** Standardize API responses so both success payloads and error states follow a consistent JSON structure.
- **Global Error Catching:** Centralize exception handling to prevent application crashes, hide internal stack traces from clients, and return appropriate HTTP status codes.

PROJECT DESIGN PHASE

1. Technical Architecture

The FitTrack AI backend follows the Model-View-Controller (MVC) architectural design pattern, augmented with a dedicated Service Layer and Middleware Layer to promote modularity, maintainability, and loose coupling between components.



Layer Breakdown & Responsibilities

- Model Layer (Data Layer):** Defines the data structure, constraints, and validation rules using Mongoose Schemas; communicates directly with MongoDB to perform persistent database operations; contains the User Schema (credentials and profile data) and the Workout Schema (activity tracking).
- View Layer (API Interface):** As a RESTful backend API, the application does not serve traditional server-side HTML UI templates. The View Layer is represented by API consumers such as React frontend web apps, mobile applications, or testing utilities like Postman / Thunder Client. Receives input via HTTP request payloads and consumes standard JSON responses.
- Controller Layer (Business Logic):** Acts as the intermediary between routes, services, and models. Validates input data, executes business rules, calls database methods or external services, and formats standard JSON responses. Includes authController, workoutController, and aiController.
- Service Layer (Reusable Modules):** Houses isolated, highly reusable business logic used across multiple controllers. Gemini AI Service prepares prompts and sends demographic or activity stats to Google Gemini API, returning structured workout plans and fitness insights. JWT Service handles signing and verifying JSON Web Tokens. Password Hashing Service manages bcrypt.js salt generation, hashing, and comparison.

2. ER-Diagram

The application backend manages data persistence through two primary collections connected via a One-to-Many relationship.

A. User Entity (users collection)

Stores registered user credentials and profile metadata.

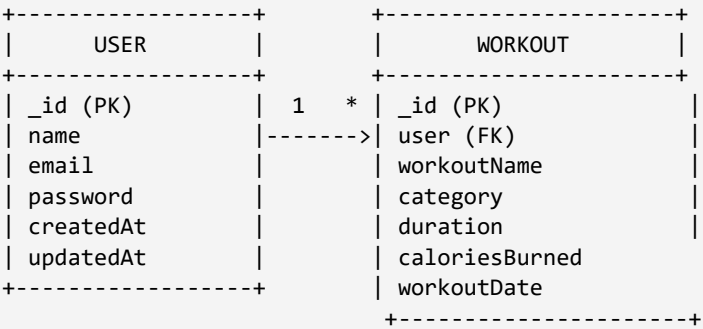
Field	Type	Constraint	Description
_id	ObjectId	Primary Key, Auto-generated	Unique document identifier
name	String	Required	Full name of the user
email	String	Required, Unique, Lowercase	User email address for authentication
password	String	Required, Encrypted (bcrypt.js)	Hashed user password
createdAt	Date	Timestamp (Auto)	Record creation timestamp
updatedAt	Date	Timestamp (Auto)	Record last update timestamp

B. Workout Entity (workouts collection)

Stores individual workout session logs logged by authenticated users.

Field	Type	Constraint	Description
_id	ObjectId	Primary Key, Auto-generated	Unique document identifier
user	ObjectId	Foreign Key (ref: 'User'), Required	References the parent User document
workoutName	String	Required	Name of exercise (e.g., "Morning Run")
category	String	Required	Category (e.g., Cardio, Strength)
duration	Number	Required, min: > 0	Exercise duration in minutes
caloriesBurned	Number	Required, min: >= 0	Total estimated calories burned
workoutDate	Date	Required	Date when the workout occurred
createdAt	Date	Timestamp (Auto)	Record creation timestamp
updatedAt	Date	Timestamp (Auto)	Record last update timestamp

C. Entity Relationship: One-to-Many (User → Workouts)



- Relationship Explanation: One User can log multiple Workout records over time. However, each Workout document belongs strictly to one parent user, enforced by referencing the user’s ObjectId in the user field.

3. Features

The key capabilities delivered by the FitTrack AI backend:

- Secure Authentication — Registration and login protected by bcrypt.js password hashing and JWT-based session tokens.
- Workout Management — Full CRUD (Create, Read, Update, Delete) for personal workout logs, scoped to the authenticated user.
- Workout Search — Dynamic filtering of workout history by keyword, category, or date.
- Google Gemini Integration — A dedicated service layer wraps the @google/genai SDK for all AI calls.
- AI Workout Recommendations — Personalized weekly routines generated from age, fitness goal, and experience level.
- AI Fitness Insights — Progress evaluation and motivational feedback generated from aggregated workout statistics.
- Centralized Error Handling — A single Express error-handling middleware standardizes every failure response.
- RESTful, Modular Architecture — Routes, controllers, services, and models are cleanly separated for maintainability and parallel development.

4. Roles and Responsibilities

The application currently defines a single application-level role — the Authenticated User — enforced by the JWT authentication middleware on every protected route.

Authenticated User

Responsibilities:

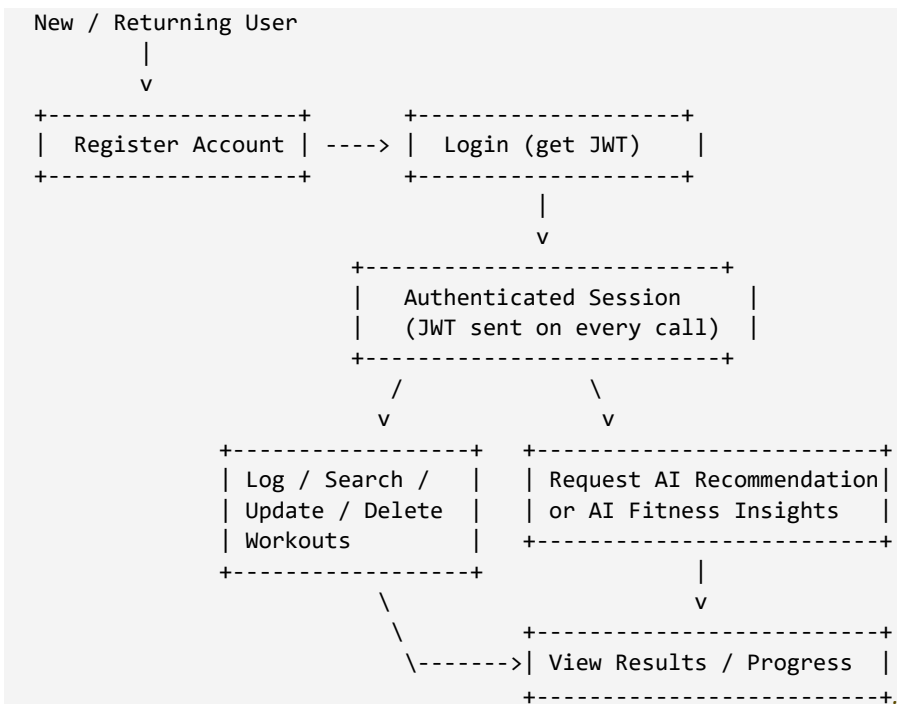
- Register an account and keep login credentials secure.
- Log workouts accurately (name, category, duration, calories, date).
- Review AI-generated recommendations and insights and apply them responsibly.

Permissions:

- Full CRUD access to their own workout records only — every query is scoped by req.user.id, so one user can never read or modify another user's data.
- Read access to their own profile via GET /api/auth/profile.
- Access to both AI endpoints (recommendation and insights) using their own workout statistics.

5. User Flow

End-to-end path a user takes through the system, from first visit to receiving an AI insight:



6. MVC Pattern

FitTrack AI applies the MVC pattern as follows, expanding on the layer breakdown above:

Model Layer (Data Layer)

- Mongoose schemas (User, Workout) define shape, types, and validation rules for every document stored in MongoDB.
- Models are the only layer that talks directly to the database.

View Layer (API Interface)

- There is no server-rendered HTML — the "view" is the JSON contract returned to API consumers (a future React client, a mobile app, or Postman during testing).
- Every response follows the standardized { success, data } / { success, error } shape defined in NFR3.

Controller Layer (Business Logic)

- authController, workoutController, and aiController each own one functional domain.
- Controllers validate input, call the Service or Model layer, and translate the result into an HTTP response — they never talk to MongoDB directly.

7. API Route Mapping

The API routes are categorized by functional domain, mapped to specific HTTP methods, and secured using the JWT authentication middleware where applicable.

A. Auth Routes (/api/auth)

Method	Route Endpoint	Access	Controller Method	Description
POST	/api/auth/register	Public	registerUser	Creates a new user account with hashed password.
POST	/api/auth/login	Public	loginUser	Verifies password & returns JWT authorization token.

Method	Route Endpoint	Access	Controller Method	Description
GET	/api/auth/profile	Protected (JWT)	getUserProfile	Retrieves profile info of the logged-in user.

B. Workout Routes (/api/workouts)

Method	Route Endpoint	Access	Controller Method	Description
POST	/api/workouts/	Protected (JWT)	addWorkout	Logs a new workout session linked to the user.
GET	/api/workouts/	Protected (JWT)	getWorkouts	Fetches all workouts belonging to the user.
GET	/api/workouts/search	Protected (JWT)	searchWorkouts	Filters workouts by keyword, category, or date.
GET	/api/workouts/:id	Protected (JWT)	getWorkoutById	Retrieves a specific workout document by ID.
PUT	/api/workouts/:id	Protected (JWT)	updateWorkout	Updates an existing workout record.
DELETE	/api/workouts/:id	Protected (JWT)	deleteWorkout	Removes a workout record permanently.

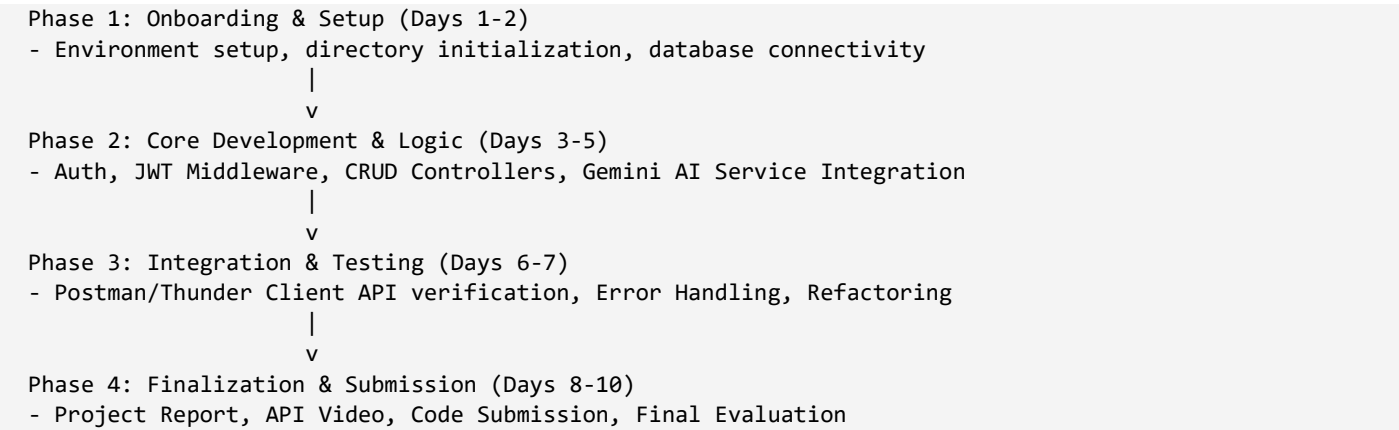
C. AI Integration Routes (/api/ai)

Method	Route Endpoint	Access	Controller Method	Description
POST	/api/ai/recommendation	Protected (JWT)	getAIRecommendation	Generates weekly exercise plan using Gemini AI.
POST	/api/ai/insights	Protected (JWT)	getFitnessInsights	Generates performance analysis from workout stats.

PROJECT PLANNING PHASE

1. Project Roadmap & Schedule

The development lifecycle of FitTrack AI follows a structured project timeline aligned with the academic and industry project submission milestones.



Milestone Breakdown

1. Milestone 1: Project Initialization & Onboarding

○ Initialize Node.js application, package installation, and version control.

○ Configure MongoDB Atlas connection and .env variables.
2. Milestone 2: Database Schema & Authentication Setup

○ Construct Mongoose Schemas for User and Workout collections.

○ Implement bcrypt.js password hashing and JWT authorization middleware.
3. Milestone 3: Core Business Logic & AI Service Integration

○ Develop RESTful CRUD endpoints and search filter controllers.

○ Integrate Google Gemini API service layer for workout recommendations and fitness insights.
4. Milestone 4: API Testing & Verification

○ Verify all route endpoints in Postman / Thunder Client.

○ Validate HTTP status codes, payload structures, and centralized error handling.
5. Milestone 5: Documentation & Submission

○ Record 5–10 minute project demonstration and API testing videos.

○ Push source code repository (excluding node_modules) and finalize project documentation.

2. Task Allocation (4-Member Team Distribution)

Responsibilities are distributed across team members based on backend architectural layers to ensure parallel development and smooth integration:

Team Member	Role	Primary Responsibilities	Deliverables
S. Gowtham	Team Leader & System Architect	Project setup, MongoDB connection configuration (db.js), Express server gateway (server.js), and .env security.	server.js, config/db.js, .env template, and master Git repo.
M. Gnanagiri	Auth & Middleware Developer	User registration/login business logic, bcrypt.js encryption, and JWT protection middleware.	models/User.js, controllers/authController.js, middleware/authMiddleware.js.
C. Giridharan	Workout CRUD & Search Engineer	Workout schema, full CRUD logic (Create, Read, Update, Delete), and query filtering by keyword/date/category.	models/Workout.js, controllers/workoutController.js, routes/workoutRoutes.js.

Team Member	Role	Primary Responsibilities	Deliverables
R. Hariharasudhan	Gemini AI Service & Quality Assurance	Integrating @google/generative-ai, prompt engineering for plans and insights, API testing in Postman, and video documentation.	services/geminiService.js, controllers/aiController.js, API testing video, and project documentation.

3. Environment Setup & Configuration Plan

Environment Variables Structure (.env)

All sensitive access keys, secrets, and environment parameters are isolated in a .env file at the root of the server folder:

```
# Application Server Port
PORT=5000

# MongoDB Connection String (Local or MongoDB Atlas)
MONGO_URI=mongodb+srv://<username>:<password>@cluster.mongodb.net/fittrack

# JWT Secret Key
JWT_SECRET=<your_jwt_secret>

# Google Gemini API Key
GEMINI_API_KEY=<your_gemini_api_key>

# Google Gemini Version
GEMINI_MODEL=gemini-3.7-flash
```

PROJECT DEVELOPMENT PHASE

1. Creating Project Folder

Steps to support the backend project structure:

1. Create the root project folder, e.g. AI-FitTrack-Backend, and open it in Visual Studio Code.
2. Inside it, create a dedicated server/ directory to hold all backend source code.
3. Inside server/, create the sub-folders: config/, controllers/, middleware/, models/, routes/, and services/.

2. Server Setup

Run the following terminal commands to initialize the Node.js environment and install required packages:

```
# 1. Initialize package.json
npm init -y

# 2. Install production dependencies
npm install express mongoose jsonwebtoken bcryptjs @google/genai dotenv cors

# 3. Install development dependency for live server auto-reload
npm install -D nodemon
```

The application entry point (server.js) wires up Express, CORS, JSON body parsing, the database connection, and every route group.

3. Backend Structure

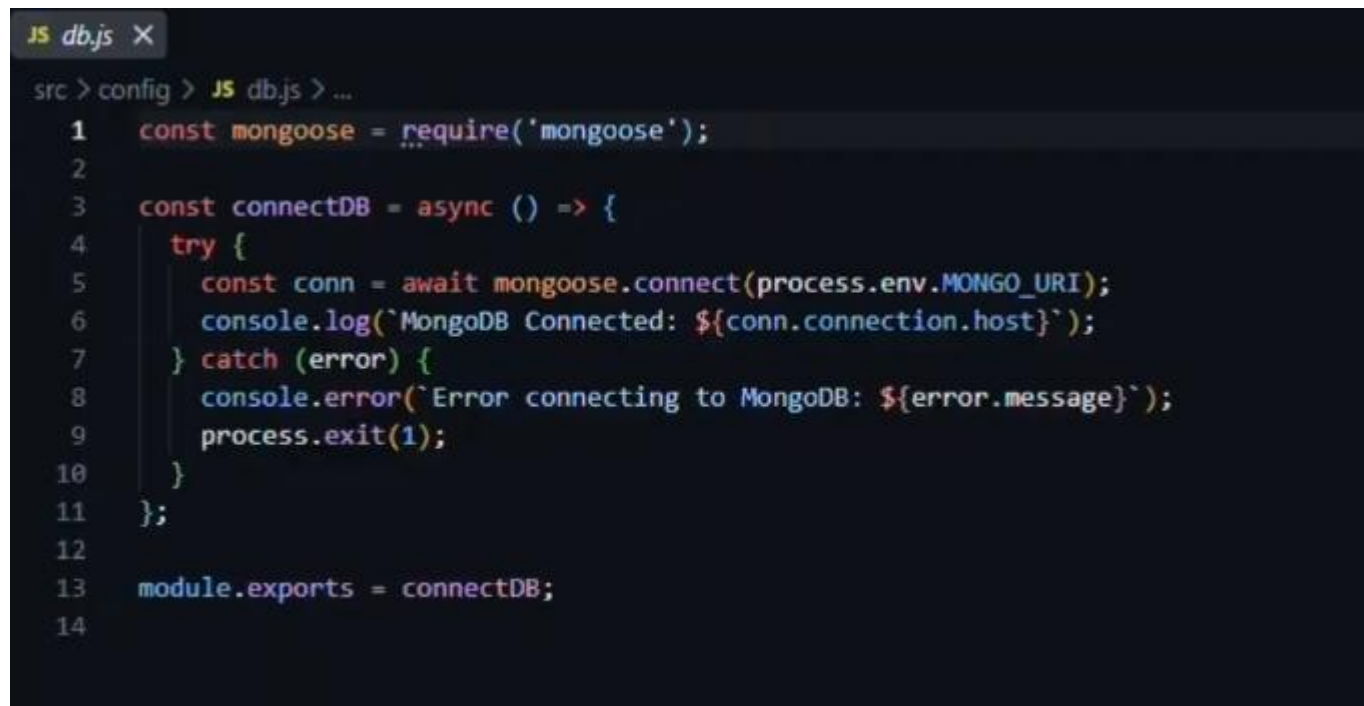
The GitHub repository organizes backend source code inside a dedicated /server directory to maintain modular separation:

```
AI-FitTrack-Backend/
|-- server/
|   |-- config/          # Database configuration (db.js)
|   |-- controllers/     # Auth, Workout, and AI business logic
|   |-- middleware/      # JWT protection and error middleware
|   |-- models/          # User and Workout Mongoose schemas
|   |-- routes/          # API route definitions
|   |-- services/        # Google Gemini AI wrapper module
|   |-- .env.example     # Environment variable template
|   |-- package.json     # Node.js dependencies and run scripts
|   `-- server.js        # Server application entry point
|-- .gitignore           # Git file exclusion rules
`-- README.md            # Master setup guide & API reference
```

4. Development and Explanation

A. Configure MongoDB & Create Database Connection (config/db.js)

Connects the Express application to a MongoDB Atlas cluster (or a local MongoDB instance) using Mongoose. The MONGO_URI is generated from the MongoDB Atlas dashboard (Database → Connect → Drivers) and stored in .env, never committed to source control.

A screenshot of a code editor with a dark theme. The file name 'db.js' is visible in the top left. The code is as follows:

```
1  const mongoose = require('mongoose');
2
3  const connectDB = async () => {
4    try {
5      const conn = await mongoose.connect(process.env.MONGO_URI);
6      console.log(`MongoDB Connected: ${conn.connection.host}`);
7    } catch (error) {
8      console.error(`Error connecting to MongoDB: ${error.message}`);
9      process.exit(1);
10   }
11 };
12
13 module.exports = connectDB;
14
```

B. Create Schema and Models

The User and Workout Mongoose schemas (shown in full under FR1 and FR2 in the Requirement Analysis Phase) define the shape, types, and validation rules for both collections. They are the only layer that communicates directly with MongoDB.

C. Security & Authentication Middleware (middleware/authMiddleware.js)

Verifies incoming JWT tokens in the HTTP Authorization header to secure private endpoints.

```
JS auth.js X
src > middleware > JS auth.js > ...
1  const jwt = require('jsonwebtoken');
2  const User = require('../models/User');
3
4  const protect = async (req, res, next) => {
5    let token;
6
7    if (
8      req.headers.authorization &&
9      req.headers.authorization.startsWith('Bearer')
10   ) {
11     try {
12       // Get token from header
13       token = req.headers.authorization.split(' ')[1];
14
15       // Verify token
16       const decoded = jwt.verify(token, process.env.JWT_SECRET);
17
18       // Get user from the token, excluding password
19       req.user = await User.findById(decoded.id).select('-password');
20
21       if (!req.user) {
22         return res.status(401).json({
23           success: false,
24           error: 'Not authorized, user not found',
25         });
26       }
27
28       return next();
29     } catch (error) {
30       console.log(error);
31       return res.status(401).json({
32         success: false,
33         error: 'Not authorized, token failed',
34       });
35     }
36   }
37   return next();
38 }
```

```

JS auth.js X
src > middleware > JS auth.js > ...
4   const protect = async (req, res, next) => {
13   // Get token from request
14   token = req.headers.authorization.split(' ')[1];
15   // Verify token
16   const decoded = jwt.verify(token, process.env.JWT_SECRET);
17   // Get user from the token, excluding password
18   req.user = await User.findById(decoded.id).select('-password');
21   if (!req.user) {
22     return res.status(401).json({
23       success: false,
24       error: 'Not authorized, user not found',
25     });
26   }
27
28   return next();
29 } catch (error) {
30   console.error('JWT verification error:', error.message);
31   return res.status(401).json({
32     success: false,
33     error: 'Not authorized, token failed',
34   });
35 }
36 }
37
38 if (!token) {
39   return res.status(401).json({

```

D. CRUD Operations Logic (controllers/workoutController.js)

Executes complete workout data management (Create, Read, Search/Filter, Update, Delete).

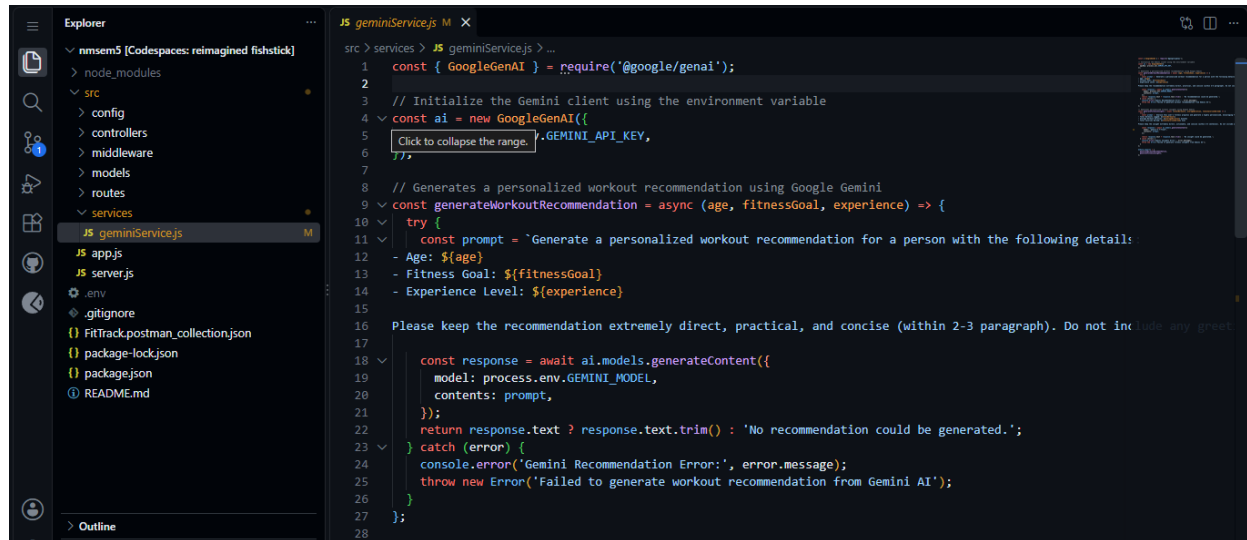
```

JS workoutController.js X
src > controllers > JS workoutController.js > ...
6   const addWorkout = async (req, res, next) => {
9
10   // Validate fields
11   if (!workoutName || !category || !duration || !caloriesBurned) {
12     return res.status(400).json({
13       success: false,
14       error: 'Please provide workoutName, category, duration, and caloriesBurned',
15     });
16   }
17
18   // Create workout record scoped to user
19   const workout = await Workout.create({
20     workoutName,
21     category,
22     duration,
23     caloriesBurned,
24     workoutDate: workoutDate || new Date(),
25     user: req.user._id,
26   });
27
28   res.status(201).json({
29     success: true,
30     data: workout,
31   });
32 } catch (error) {
33   next(error);
34 }
35 }

```

E. Gemini AI Integration Service (services/geminiService.js)

Constructs structured prompt payloads and extracts text outputs from @google/genai.



The screenshot shows a Visual Studio Code editor with a project named 'nmssem5 [Codespaces: reimagined fishstick]'. The Explorer sidebar on the left shows the file structure, with 'services/geminiService.js' selected. The main editor displays the code for 'geminiService.js'. The code imports 'GoogleGenAI' from '@google/genai', initializes the client using an environment variable 'GEMINI_API_KEY', and defines an asynchronous function 'generateWorkoutRecommendation'. This function constructs a prompt for generating a personalized workout recommendation based on age, fitness goal, and experience. It then uses the 'ai.models.generateContent' method to get a response and returns the text, with error handling for failures.

```
src > services > JS geminiService.js > ...
1  const { GoogleGenAI } = require('@google/genai');
2
3  // Initialize the Gemini client using the environment variable
4  const ai = new GoogleGenAI({
5    GEMINI_API_KEY,
6  });
7
8  // Generates a personalized workout recommendation using Google Gemini
9  const generateWorkoutRecommendation = async (age, fitnessGoal, experience) => {
10   try {
11     const prompt = `Generate a personalized workout recommendation for a person with the following details:
12     - Age: ${age}
13     - Fitness Goal: ${fitnessGoal}
14     - Experience Level: ${experience}
15
16     Please keep the recommendation extremely direct, practical, and concise (within 2-3 paragraph). Do not include any greet
17
18     const response = await ai.models.generateContent({
19       model: process.env.GEMINI_MODEL,
20       contents: prompt,
21     });
22     return response.text ? response.text.trim() : 'No recommendation could be generated.';
23   } catch (error) {
24     console.error('Gemini Recommendation Error:', error.message);
25     throw new Error('Failed to generate workout recommendation from Gemini AI');
26   }
27 };
28
```

Fig. Gemini AI service implementation (services/geminiService.js)

PROJECT TESTING PHASE

1. API Verification Workflow (Postman / Thunder Client)

API verification ensures that each RESTful endpoint operates correctly according to system specifications, enforces route authorization, and returns standard JSON payloads. Sequential testing is conducted using Postman or the Thunder Client VS Code extension.

A. Authentication Flow

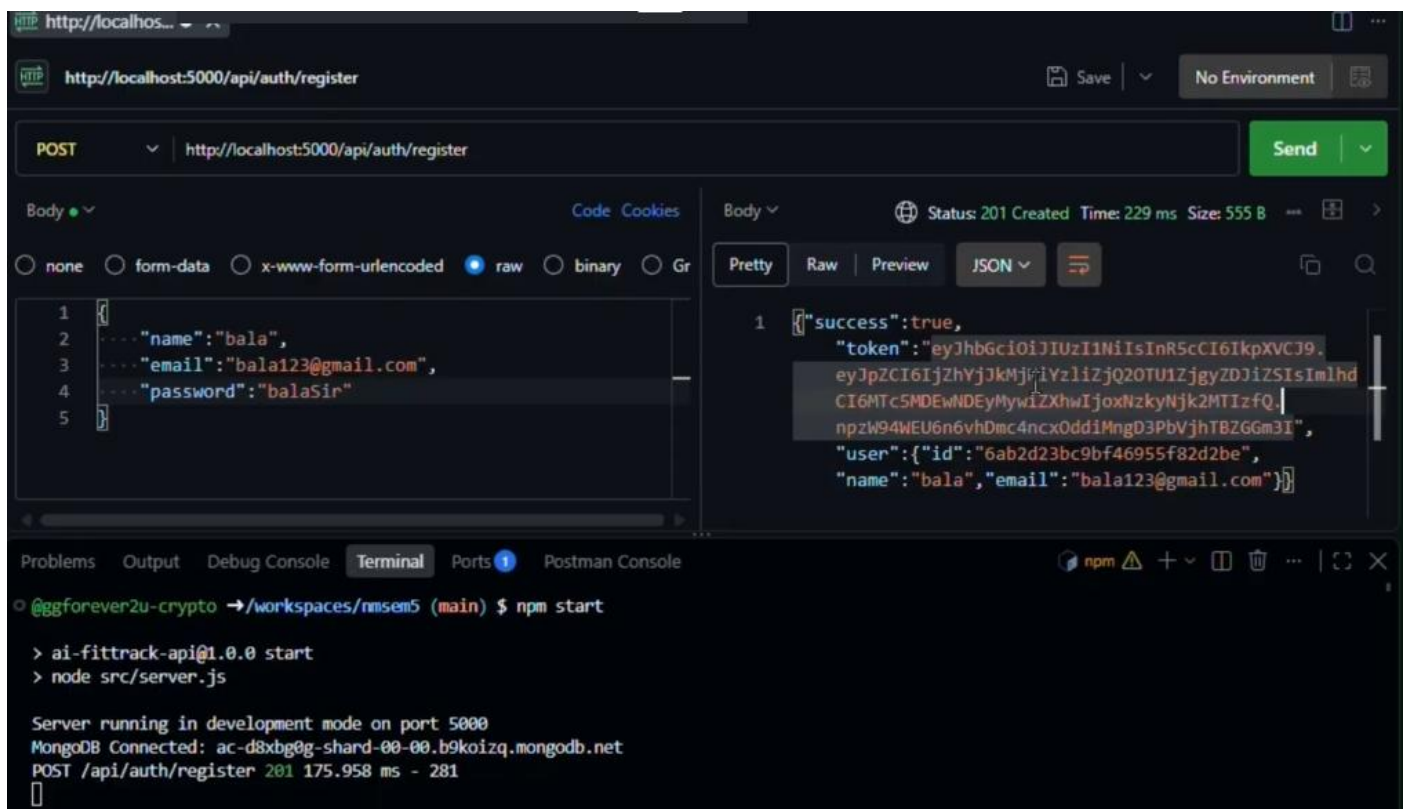
1. POST /api/auth/register

- Purpose: Registers a new user with encrypted password storage.
- Request Payload:

```
{
  "name": "Rahul Sharma",
  "email": "rahul@fittrack.com",
  "password": "Password123"
}
```

- Expected Response (201 Created):

```
{
  "success": true,
  "message": "User registered successfully",
  "data": {
    "_id": "66f41b2a9e3d4c0012a4b891",
    "name": "Rahul Sharma",
    "email": "rahul@fittrack.com"
  }
}
```



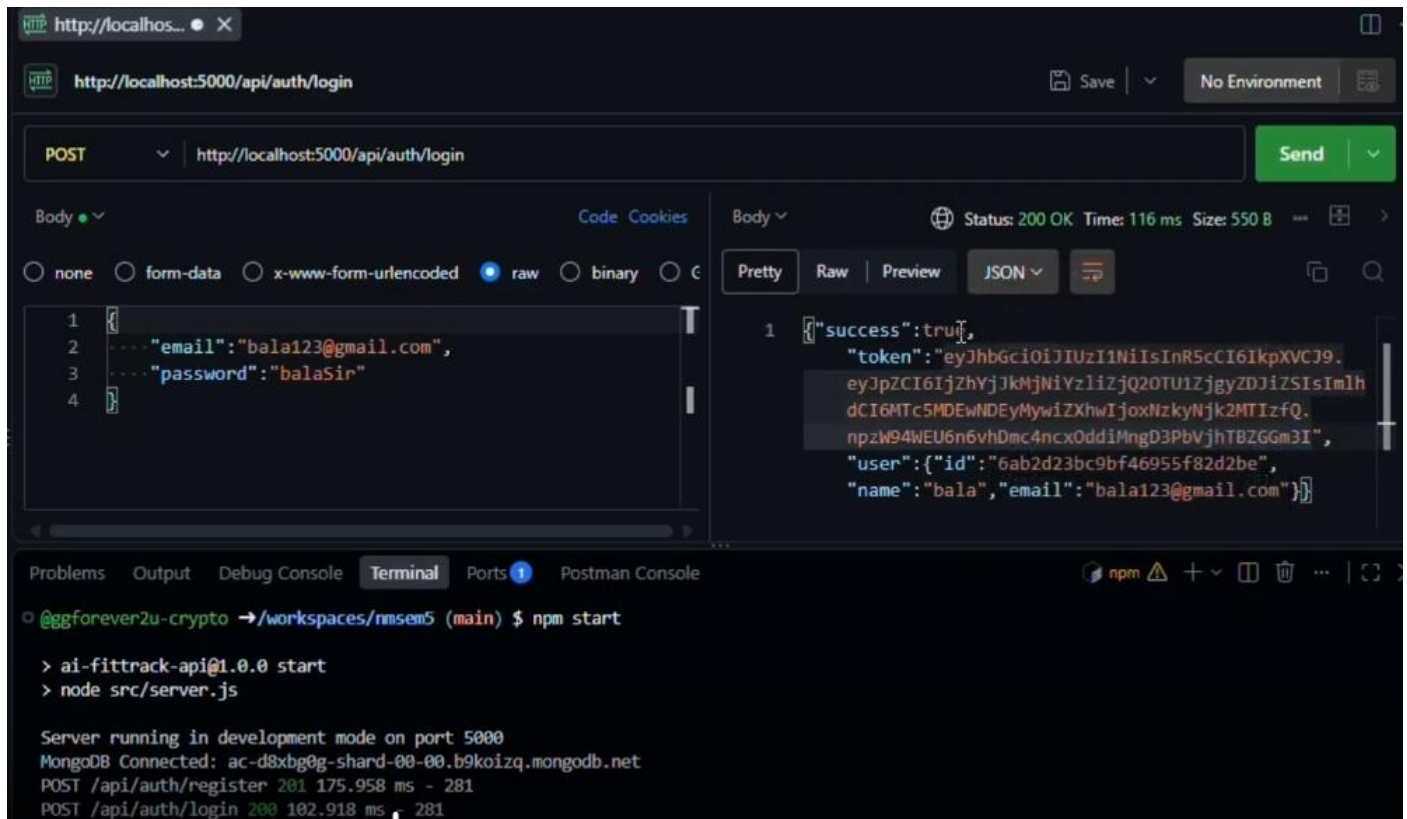
2. POST /api/auth/login

- Purpose: Authenticates user credentials and issues a signed JWT bearer token.
- Request Payload:

```
{
  "email": "rahul@fittrack.com",
  "password": "Password123"
}
```

- Expected Response (200 OK):

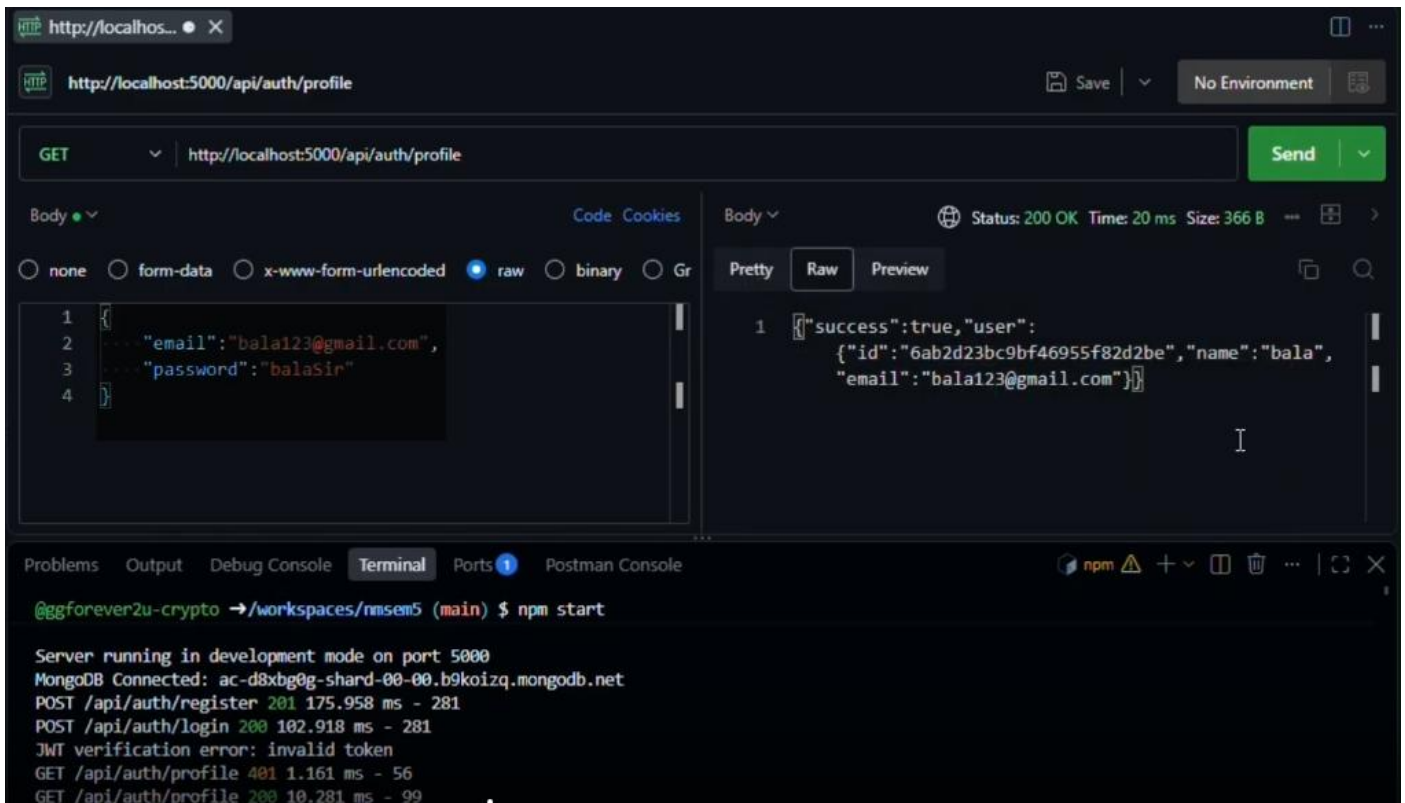
```
{
  "success": true,
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```



- ### 3. GET /api/auth/profile

- Purpose: Retrieves authenticated profile details using an Authorization: Bearer <token> header.
- Expected Response (200 OK):

```
{
  "success": true,
  "data": {
    "_id": "66f41b2a9e3d4c0012a4b891",
    "name": "Rahul Sharma",
    "email": "rahul@fittrack.com"
  }
}
```



B. Workout Management Flow

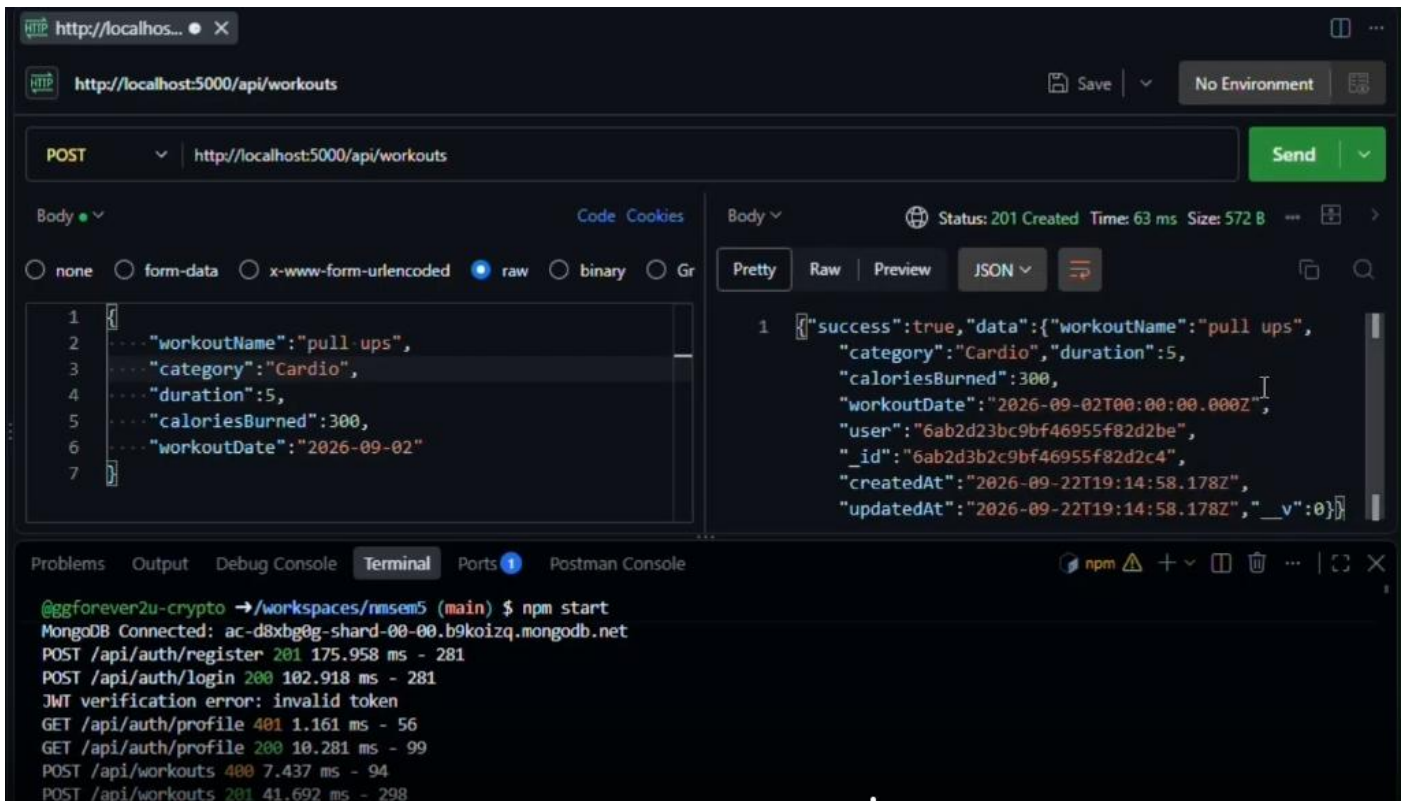
1. POST /api/workouts/

- Purpose: Logs a new workout session for the authenticated user.
- Request Payload:

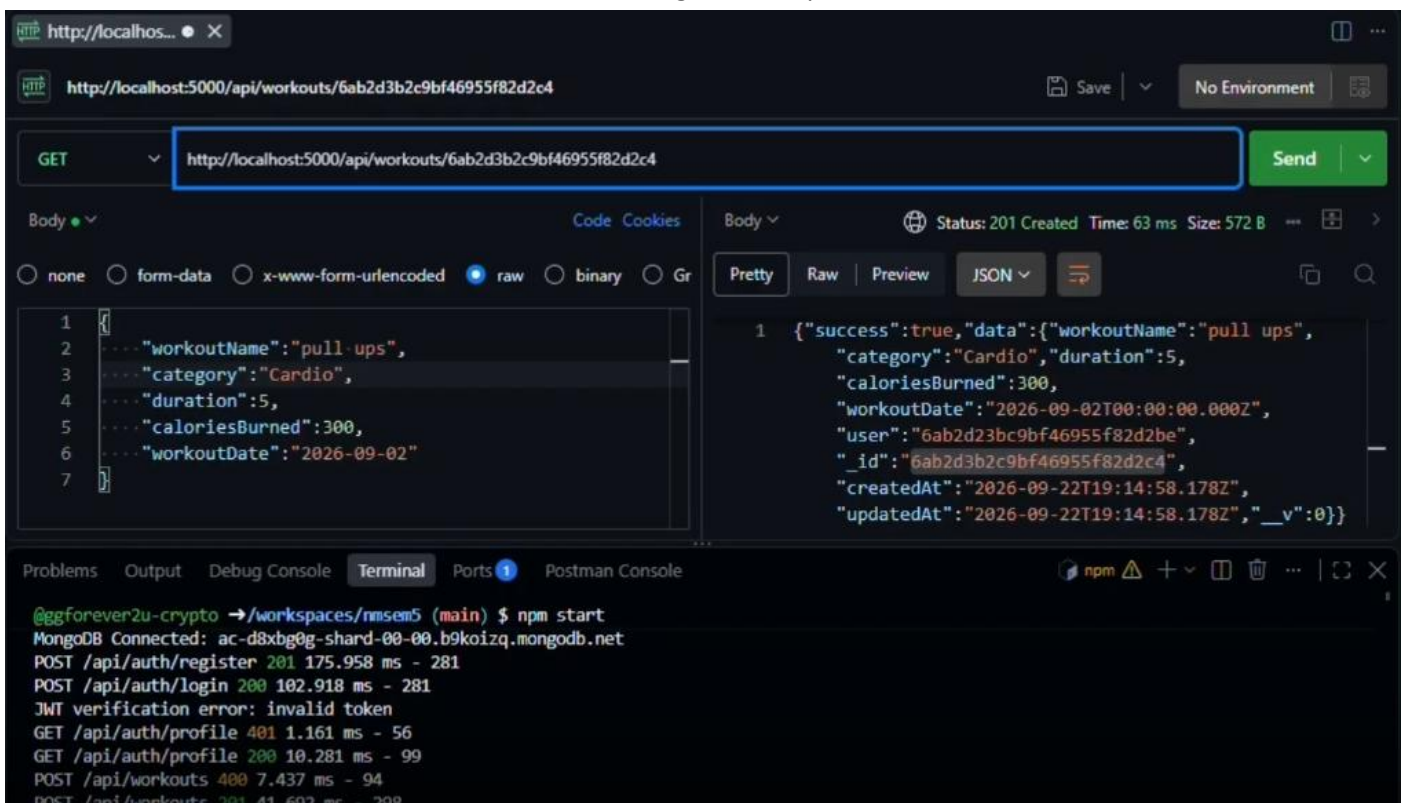
```
{
  "workoutName": "Morning Run",
  "category": "Cardio",
  "duration": 30,
  "caloriesBurned": 250,
  "workoutDate": "2026-09-20"
}
```

- Expected Response (201 Created):

```
{
  "success": true,
  "data": { "_id": "670a...", "workoutName": "Morning Run", "category": "Cardio", "duration": 30,
  "caloriesBurned": 250 }
}
```



2. GET `/api/workouts/search?keyword=run&category=Cardio`
 - Purpose: Filters the logged-in user's workouts by keyword and/or category.
 - Expected Response (200 OK): an array of matching workout documents.
3. PUT `/api/workouts/:id` and DELETE `/api/workouts/:id`
 - Purpose: Updates or permanently removes a specific workout owned by the authenticated user; returns 404 if the record does not belong to the requester.



C. AI Integration Flow

1. POST /api/ai/recommendation

o Request Payload:

```
{
  "age": 24,
  "fitnessGoal": "Muscle Gain",
  "experienceLevel": "Beginner"
}
```

o Expected Response (200 OK): { "recommendation": "<AI-generated weekly plan text>" }

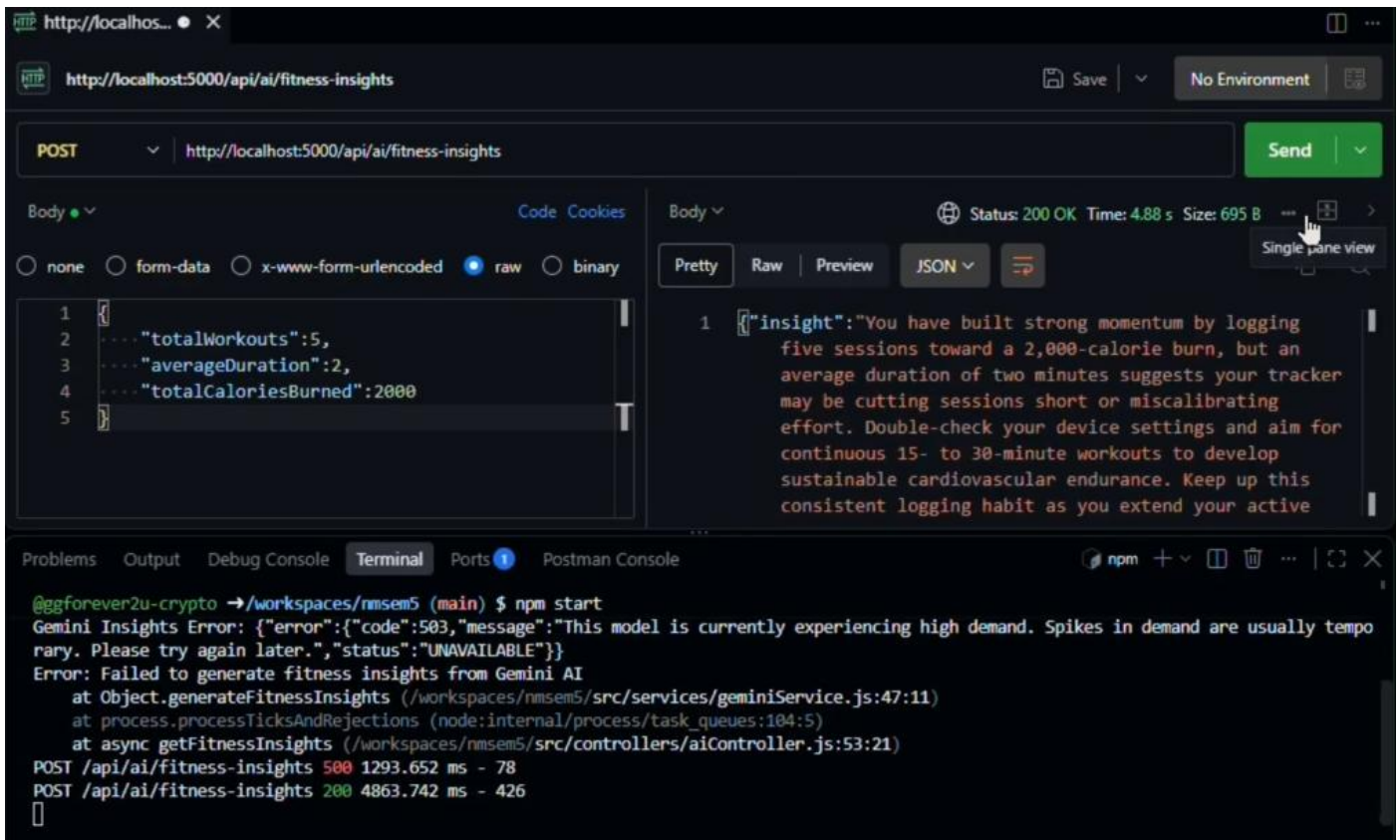
The screenshot shows a web browser with the URL `http://localhost:5000/api/ai/workout-recommendation`. The request method is `POST`. The request body is a JSON object: `{ "age": 19, "fitnessGoal": "muscle mass gain", "experience": "one year gym experience" }`. The response status is `200 OK` with a time of `6.33 s` and a size of `1.83 KB`. The response body is a detailed workout plan. Below the browser, a terminal window shows the command `npm start` and the output of the application, including an error message about high demand from Gemini AI.

2. POST /api/ai/insights

o Request Payload:

```
{
  "totalWorkouts": 12,
  "averageDuration": 35,
  "totalCaloriesBurned": 3600
}
```

o Expected Response (200 OK): { "insights": "<AI-generated performance summary>" }



2. Error & Edge Case Verification

- Missing / invalid Authorization header on a protected route returns 401 with a clear message.
- Duplicate email on registration is rejected by the unique index on User.email.
- Requesting or modifying another user's workout by ID returns 404, never another user's data.
- All unhandled exceptions fall through to the centralized error-handling middleware (NFR3), keeping the response shape consistent.

PROJECT DOCUMENTATION PHASE

1. Phase Status Sheets (PSS) Compilation

The Phase Status Sheets (PSS) consolidate all technical descriptions, architectural designs, and system workflows into the official project documentation template:

- **Technical Descriptions:** Documents system goals, modular MVC architecture, JWT authentication flow, Mongoose ORM database integration, and Google Gemini AI service logic.
- **Entity-Relationship (ER) Diagram:** Details the One-to-Many (User → Workouts) database model, primary keys (`_id`), foreign key references (`user`), attribute data types, and validation constraints.
- **Data Flow Diagrams (DFD) & User Flow:** Charts the flow of incoming HTTP requests from API consumers through Express routes, JWT auth middleware, controllers, database models, and external Google Gemini API services.
- **Document Integration:** Assembles individual phase outputs — from initial Brainstorming & Ideation through Testing — into the final PSS report template.

2. Source Code Repository Setup

A. Clean GitHub Repository Structure

The GitHub repository organizes backend source code inside a dedicated `/server` directory to maintain modular separation (see the Backend Structure diagram in the Project Development Phase).

B. Strict File Exclusion (`.gitignore`)

To avoid committing bulky build dependencies and leaking confidential security keys, the `.gitignore` file strictly excludes `node_modules/` and local `.env` configuration files:

```
# Dependency directory - MUST BE EXCLUDED
node_modules/

# Environment and secret keys
.env
.env.local

# Logs and runtime data
npm-debug.log*
yarn-debug.log*

# OS metadata
.DS_Store
Thumbs.db
```

C. Project README.md Documentation

FitTrack AI - Backend RESTful API

FitTrack AI is a secure, AI-augmented RESTful backend built using Node.js, Express.js, MongoDB, and Google Gemini AI.

Prerequisites

- Node.js: v16.x or higher
- npm: v8.x or higher
- MongoDB: Local MongoDB instance or MongoDB Atlas URI
- Google Gemini API Key: Obtained from Google AI Studio

Installation & Setup

1. Clone the Repository:

```
git clone https://github.com/gnna-m/fittrack.git  
cd fittrack/server
```

2. Install Dependencies:

```
npm install
```

3. Configure Environment Variables:

Create a .env file inside the /server folder (see Environment Setup & Configuration Plan, Project Planning Phase).

4. Run the Server:

```
npm run dev
```

***PROJECT
DEMONSTRATION PHASE***

1. Video Deliverables Specification & Links

To meet the mandatory project submission criteria, two separate video recordings have been created, uploaded to Google Drive, and configured with Public Access ("Anyone with the link can view"):

Backend Overview & Code Walkthrough Video (7–10 minutes)

- Demonstration Coverage: Shows local server initialization using `npm run dev / nodemon`, successful MongoDB Atlas database connectivity, Visual Studio Code workspace structure (MVC architecture), environment variable configuration (`.env`), and code logic explanation for controllers and the Google Gemini AI service wrapper.
- Public Link: <https://drive.google.com/file/d/13GZndDkmLn-InRGoQyWNY9swqQqutYAo/view?usp=drivesdk>

API Testing & Verification Video

Demonstration Coverage: Demonstrates live sequential endpoint execution inside Postman or Thunder Client:

1. User Authentication: Registration (POST /register), Login (POST /login), and acquisition of the JWT Bearer token.
 2. Protected Routes: Attaching Authorization: Bearer <token> to every subsequent request and confirming access is granted.
 3. Workout CRUD: Creating, listing, searching, updating, and deleting a workout record.
 4. AI Endpoints: Calling `/api/ai/recommendation` and `/api/ai/insights` and reviewing the generated text.
- Public Link: [api testing video](#)